

2일차: 고가용성 아키텍처 설계 및 모니터링 실습

1. 고가용성 아키텍처 설계

고가용성(High Availability)은 서비스 연속성을 보장하기 위한 핵심 설계 원칙입니다. AWS 및 GCP 모두 물리적으로 분리된 인프라를 활용해 장애에 대비하는 아키텍처를 제공합니다. **AWS**에서는 가용 영역(AZ) 단위로 이중화하고, **GCP**에서는 지역(Region) 단위로 이중화합니다. 이를 통해 한 구역이나 한 리전에서 장애가 발생해도 다른 구역/리전에서 서비스가 지속되도록 설계합니다.

- **AWS Multi-AZ 구성:** Amazon RDS 같은 관리형 데이터베이스는 **동기 복제**를 통해 서로 다른 AZ에 대기 인스턴스를 두고 장애 조치를 수행합니다. 예를 들어, RDS 다중 AZ 배포는 “데이터 손실 없이 수동 개입 없이 60초 이내에 장애 조치”하도록 자동화되어 있어, 애플리케이션의 고가용성을 지원합니다 ¹. 즉, 프라이머리 DB 인스턴스의 모든 데이터가 두 번째 AZ의 대기 인스턴스로 실시간 동기 복제되며, 장애 발생 시 자동으로 대기 인스턴스가 프라이머리 역할을 이어받습니다 ¹. 다음 그림은 RDS 다중 AZ 배포의 동작 방식을 보여줍니다. 각 AZ에 독립적으로 위치한 데이터베이스를 운영함으로써 하나의 AZ 장애에도 시스템 전체의 지속성을 확보합니다.

그림 1: Amazon RDS 다중 AZ 구성 예시 (AWS 공식 문서)

- **실습 (AWS RDS 다중 AZ):** AWS 콘솔 또는 CLI로 MySQL/MariaDB/Aurora 등의 RDS 인스턴스를 생성할 때 **Multi-AZ 배포 옵션**을 선택합니다. 예를 들어 AWS CLI에서는 `aws rds create-db-instance --db-instance-identifier mydb --engine mysql --db-instance-class db.t3.micro --allocated-storage 20 --master-username admin --master-user-password mypassword --multi-az` 명령으로 다중 AZ DB 인스턴스를 생성할 수 있습니다. 생성 후 AWS 콘솔에서 [데이터베이스 인스턴스 → 작업 → 장애 조치 시뮬레이션]을 통해 실제 장애 조치(failover)를 테스트해볼 수 있습니다. 장애 조치가 완료되면 **60초 이내에** 대기 인스턴스가 프라이머리로 전환되며 정상 연결이 유지되는 것을 확인할 수 있습니다.
- **오류 해결 팁:** RDS 다중 AZ 생성 시, 비밀번호나 인스턴스 클래스 제약, VPC 설정 오류가 자주 발생합니다. 비밀번호는 영문·숫자 조합 8~41자여야 하며, VPC 내에서 서브넷 그룹과 보안 그룹이 올바르게 구성되었는지 확인하세요. 장애 조치가 자동으로 진행되지 않으면 CloudWatch Logs나 이벤트 탭에서 실패 원인을 확인합니다.
- **AWS EC2 인스턴스 다중 AZ 구성:** 일반 서버(VM)로 구성할 때에도 여러 AZ에 걸쳐 인스턴스를 배치해야 합니다. 예를 들어 Node.js 애플리케이션 서버를 EC2로 운영한다면, 서로 다른 AZ의 서브넷에 복수의 인스턴스를 생성하고 이를 **Application Load Balancer (ALB)** 뒤에 둡니다. ALB는 “하나 이상의 AZ에 있는 여러 대상(EC2 인스턴스)으로 들어오는 트래픽을 자동으로 분산”해 주므로, 특정 AZ가 단절되어도 다른 AZ의 인스턴스가 요청을 처리합니다 ².
- **실습 (AWS EC2 + ALB 다중 AZ):** AWS CLI 또는 콘솔로 2개 이상의 EC2 인스턴스를 서로 다른 AZ의 서브넷에 생성합니다. 예를 들어 각 AZ별로 `aws ec2 run-instances` 명령을 실행하여 Ubuntu/Amazon Linux 인스턴스를 띄우고, 여기에 간단한 Node.js 웹 서버(예: `app.js`)를 배포합니다. 이후 `aws elbv2 create-load-balancer --name my-alb --subnets subnet-aaa subnet-bbb`로 ALB를 생성하고, `aws elbv2 create-target-group` 및 `aws elbv2 register-targets`로 EC2 인스턴스를 타겟 그룹에 등록합니다. 클라이언트는 ALB의 DNS 이름으로 접속해 요청을 보내면, ALB가 백엔드 인스턴스로 자동으로 분산 처리합니다.

- 예상 결과: ALB 앞에서 `curl http://<ALB DNS>` 를 반복 실행하면, 각각 다른 EC2 인스턴스의 Node.js 서버로 요청이 분산되는 것을 볼 수 있습니다. 한 AZ의 인스턴스를 의도적으로 종료(shutdown)하면, ALB가 자동으로 감지하여 해당 AZ의 요청을 다른 AZ로 전환합니다.
- 오류 해결 팁: ALB 구성 시 보안 그룹과 서브넷 설정이 흔히 문제됩니다. ALB와 EC2 인스턴스 보안 그룹에 HTTP/HTTPS 포트 허용 규칙(인바운드 80 또는 443) 및 헬스체크 포트 허용을 확인하세요. 또한 EC2 인스턴스에 실행 중인 Node.js 앱이 올바른 포트에 바인딩되어 있는지(`app.listen(80)` 등) 확인해야 합니다.
- **GCP 다중 리전 구성:** GCP에서는 애플리케이션을 둘 이상의 리전에 분산 배포함으로써 리전 장애에도 대응할 수 있습니다. 구글의 참조 아키텍처 문서에 따르면, 두 개 이상의 리전에서 액티브-액티브 방식으로 애플리케이션을 실행하고 **글로벌 로드 밸런서(HTTP(S) Load Balancer)** 를 이용해 트래픽을 분산시키는 구조가 권장됩니다 ³ . 이렇게 하면 각 리전 내에서 개별적으로 3개 이상의 AZ(Zone)에 인스턴스를 두고, 글로벌 로드 밸런서가 전 세계에서 가장 가까운 리전으로 요청을 라우팅합니다. 단일 Anycast IP를 사용하여 전 지역에 서비스를 노출하며, 한 리전이 완전히 장애가 나도 자동으로 다른 리전으로 트래픽을 전달해 가용성을 유지합니다 ⁴ .

그림 2: Google Cloud 글로벌 로드 밸런서를 활용한 다중 리전 애플리케이션 아키텍처 (GCP 아키텍처 센터 예시)

- 실습 (GCP Compute Engine 다중 리전 서버 배포): 먼저 두 개 이상의 리전에 인스턴스 템플릿과 인스턴스 그룹(Managed Instance Group, MIG)을 생성합니다. 예를 들어 `gcloud compute instance-templates create node-template --machine-type=e2-medium --image-family=ubuntu-2004-lts --image-project=ubuntu-os-cloud --metadata=startup-script='#!/bin/bash\napt-get update\napt-get install -y nodejs\n(코드 가져와 실행)'` 와 같이 템플릿을 만들고, `gcloud compute instance-groups managed create web-mig-a --base-instance-name web-a --size=1 --template=node-template --zones=us-central1-a,us-central1-b` 등으로 지역 내 여러 AZ에 배포합니다. 이후 **글로벌 HTTP 로드 밸런서**를 구성합니다. `gcloud compute addresses create global-ip --global` 로 글로벌 IP를 예약하고, `gcloud compute health-checks create http hc --port 80`, `gcloud compute backend-services create web-backend --protocol=HTTP --health-checks hc --global` 를 차례로 실행합니다. 각 인스턴스 그룹을 백엔드 서비스에 추가(`gcloud compute backend-services add-backend web-backend --instance-group web-mig-a --instance-group-zone us-central1-a --global` 등)하고, `gcloud compute url-maps create web-map --default-service web-backend`, `gcloud compute target-http-proxies create web-proxy --url-map web-map`, `gcloud compute forwarding-rules create http-rule --global --target-http-proxy web-proxy --ports 80` 순으로 설정하면 됩니다.
- 예상 결과: 생성된 글로벌 IP 주소로 접속하면 `#include <Region_Name>` 식으로 응답이 오거나, 두 리전의 Node.js 서버 중 가까운 쪽에서 응답이 이루어집니다. 한 리전의 인스턴스를 모두 중지시키면, 글로벌 로드 밸런서가 나머지 리전으로 자동 페일오버되어 서비스 장애 없이 트래픽을 처리합니다.
- 오류 해결 팁: GCP 로드 밸런서 구성 시 방화벽 룰을 확인하세요. HTTP(80) 및 HTTPS(443)를 허용하는 **방화벽 규칙**을 만들어야 외부에서 접근이 가능합니다. 또한, GCP CLI 명령 실행 중 리소스 부족(quota)이나 IAM 권한 부족 문제가 있을 수 있으니 적절한 프로젝트와 권한(IAM `Compute Network Admin` 등)을 확인합니다.
- **재해 복구(DR) 전략:** 고가용성과 함께 재해 복구도 고려해야 합니다. 이를 위해 RTO(Recovery Time Objective, 복구 목표 시간)와 RPO(Recovery Point Objective, 복구 시점 목표)를 정의하여 복구 성능을 정량화합니다 ⁵ . 일반적으로 멀티 AZ 구성은 같은 리전 내 장애(예: 전원, 네트워크 단절)에 대해 빠른 RTO를 제공합니다. “멀티 AZ 전략”은 동일 리전 내에서 여러 AZ에 리소스를 분산하여 **하나의 장애 원인이 두 개 이상의 AZ에 영향을 미칠 위험을 크게 줄인다고** AWS 공식 블로그가 설명합니다 ⁶ . 반면, 리전 전체가 오프라인되

는 경우에는 “멀티 리전 전략”이 필요합니다. 멀티 리전 아키텍처는 광범위한 재해 상황(자연재해, 대규모 정전 등)을 극복할 수 있어 **더 높은 가용성**을 제공합니다 ⁷ . 예를 들어 Active/Active 방식으로 두 개 이상의 리전에 걸쳐 서비스를 운영하면, 한 리전이 실패해도 다른 리전이 즉시 트래픽을 이어받습니다 ⁸ ⁹ .

재해 복구 전략은 또한 Active/Passive(주/대기) 방식과 Active/Active(다중 활성) 방식으로 구분할 수 있습니다. Active/Passive 구성에서는 평상시 한 리전만 서비스하며, 재해 시 대기 리전으로 장애 조치합니다. Active/Active 구성에서는 여러 리전이 동시에 서비스를 처리하며, 부하를 분산합니다 ¹⁰ . 설계 시에는 대상 시스템의 RTO/RPO 목표, 비용, 복구 작업 난이도 등을 종합 검토해야 합니다.

2. 로드 밸런싱 및 Auto Scaling

서비스의 가용성과 성능을 높이려면 트래픽 분산과 수평 확장이 필수적입니다. 이를 위해 AWS와 GCP는 각각 다양한 로드 밸런서와 오토스케일링 기능을 제공합니다.

- **AWS ELB (Elastic Load Balancing):** AWS ELB는 애플리케이션 계층(ALB)과 네트워크 계층(NLB) 로드 밸런서를 지원합니다. 기본적으로 ELB는 “네트워크 트래픽을 분산하여 애플리케이션 확장성을 개선”해 줍니다 ¹¹ . 특히 ALB는 HTTP/HTTPS 레벨에서 호스트/경로 기반 라우팅, SSL 종료 등의 고급 기능을 제공하며, 여러 AZ에 걸친 EC2 인스턴스로 트래픽을 분산합니다.
- **실습 (ALB 설정):** AWS 콘솔의 EC2 서비스 → 로드 밸런서 메뉴에서 Application Load Balancer를 생성합니다. 로드 밸런서 설정 시 반드시 복수의 AZ에 연관된 서브넷을 선택해 고가용성을 확보합니다. 생성 후 **Target Group**을 정의하고 앞에서 만든 EC2(Node.js) 인스턴스들을 등록합니다. ALB의 보안 그룹에는 80/443 포트를 열고, EC2 보안 그룹에도 해당 트래픽이 허용되어야 합니다. ALB DNS로 접속하여 각 인스턴스로 요청이 정상 분산되는지 확인합니다.
- **예상 결과:** 예를 들어 EC2마다 다른 응답을 반환하도록 Node.js 서버를 설정해두었다면, ALB 엔드포인트에 접속할 때마다 다른 서버에서 응답이 오는 것을 볼 수 있습니다. AWS 콘솔의 로드 밸런서 상세 정보에서 **모니터링** 탭을 보면 헬스 체크 결과와 연결 요청/응답률 등의 메트릭을 실시간으로 확인할 수 있습니다.
- **오류 해결 팁:** 만약 ALB가 백엔드 인스턴스에 연결하지 못하면, 헬스체크 상태가 **unhealthy**로 표시됩니다. 이 경우, 백엔드 인스턴스의 보안 그룹, 헬스체크 경로(URL) 및 포트 설정, 인스턴스 내부의 서비스 실행 상태 등을 점검합니다. HTTP 상태 403/504 등의 오류는 보안 그룹 또는 라우팅 문제일 수 있습니다.
- **GCP Load Balancer:** GCP의 로드 밸런서는 글로벌/리전 지원, L7/L4 로드 밸런싱, 내부/외부 로드 밸런싱 등 다양한 형태를 갖추고 있습니다. 특히 글로벌 외부 HTTP(S) 로드 밸런서는 단일 Anycast IP 주소를 프론트엔드로 사용하여 여러 지역의 백엔드로 트래픽을 분산합니다. Google에 따르면 Cloud Load Balancing은 “단일 anycast IP 주소… 여러 지역 백엔드에 대한 교차 지역 부하 분산과 자동 다중 지역 장애 조치”를 지원하며, 사용량이 급증해도 빠르게 확장할 수 있습니다 ⁴ ¹² .
- **실습 (GCP HTTP(S) Load Balancer 설정):** 앞서 생성한 다중 리전 인스턴스 그룹(MIG)을 백엔드로 사용하는 글로벌 HTTP 로드 밸런서를 구성합니다. **gcloud** 명령어를 사용하여 Frontend, Backend, URL 맵 등을 설정하거나, GCP 콘솔의 **네트워킹 > 부하 분산** 메뉴에서 **HTTP(S) 부하 분산기**를 단계별로 생성할 수 있습니다. **Frontend Configuration**에서 글로벌 IP를 선택하고, **Backend Configuration**에서 앞서 만든 인스턴스 그룹들을 추가합니다. 이때 지능형 라우팅 규칙(경로 기반 라우팅)도 설정할 수 있습니다.
- **예상 결과:** 글로벌 부하 분산기가 설정되면 생성된 외부 IP(또는 지정한 도메인)로 접근 시, 가장 가까운 리전의 서버 또는 오히려 부하가 낮은 리전의 서버로 자동 연결됩니다. 해외에서 접속 테스트를 하거나, **curl**로 리전별 응답을 비교하여 글로벌 라우팅이 동작하는지 확인합니다.
- **오류 해결 팁:** 리전 간 로드 밸런싱이 동작하지 않을 경우, Backend 서비스에 해당 인스턴스 그룹이 올바르게 등록되어 있는지 확인하세요. Backend 서비스의 헬스체크가 통과해야 트래픽을 분산합니다. 또한 GCP 로드 밸런서도 방화벽 규칙(예: **default-allow-http**)이 필요합니다.

- **Auto Scaling (AWS):** Auto Scaling 그룹(ASG)을 사용해 EC2 인스턴스를 수평 확장/축소할 수 있습니다. ASG는 런치 구성(또는 템플릿)으로 인스턴스 설정을 정의한 후, 최소/최대/현재 용량과 스케일링 정책을 지정합니다. 스케일링 정책에는 CPU 사용률, 네트워크 트래픽 등 **CloudWatch 메트릭**을 기반으로 동작하는 룰을 설정할 수 있습니다. 예를 들어 “CPU 사용률이 70% 이상이면 인스턴스 1개 추가” 같은 단순 임계치(단계별 스케일링) 정책이나, 목표 사용률(target tracking) 기반 정책을 적용합니다.

- 실습 (ASG 구성 및 테스트): AWS 콘솔 → Auto Scaling 그룹 생성에서 런치 템플릿/구성을 선택하고, 앞서 배운 인스턴스와 동일한 AMI, 사용자 데이터 스크립트(예: Node.js 서버 설치)를 지정합니다. 최소/최대 용량(예: 1~3)과 트래픽을 받는 로드 밸런서를 연동합니다. 스케일 아웃 조건으로 CPU $\geq 60\%$ 를 2분 지속 시 1대 추가, 스케일 인 조건으로 CPU $< 30\%$ 를 5분 지속 시 1대 축소 같은 정책을 만듭니다.

- 예상 결과: 인스턴스 수가 1대인 상태에서 HTTP 요청을 급격히 증가시키면 (예: `ab -n 10000 -c 100 http://<ALB 주소>`), 약 1~2분 뒤 인스턴스 수가 자동으로 늘어나는 것을 확인할 수 있습니다. 반대로 트래픽이 줄면 ASG가 자동으로 인스턴스를 축소합니다. AWS 콘솔의 **Auto Scaling 그룹** 모니터링 탭에서 현재 인스턴스 개수 변화를 실시간 그래프로 볼 수 있습니다.

- 오류 해결 팁: 인스턴스가 늘어나지 않을 때는 런치 구성 오류, 용량 제한(quota), 인스턴스 부족(용량 부족) 등이 원인일 수 있습니다. CloudWatch Alarm이 제대로 생성되었는지도 확인하세요. 때로는 인스턴스 시작 시 `UserData` 스크립트가 실패하여 서버가 기동되지 않고 헬스체크에서 실패할 수도 있습니다. 이 경우 EC2 시스템 로그를 확인해 스크립트 오류를 찾아 수정해야 합니다.

- **Auto Scaling (GCP):** GCP에서는 **Managed Instance Group (MIG)**에 자동 확장 기능을 적용합니다. 각 리전/존의 인스턴스 그룹에 대해 CPU 사용률, 부하 밴더, 스케줄 기반 등 다양한 확장 기준을 설정할 수 있습니다. 예를 들어 `gcloud compute instance-groups managed set-autoscaling web-mig-a --max-num-replicas=5 --target-cpu-utilization=0.6 --cool-down-period=60 --zone=us-central1-a`로 CPU 60%를 목표로 자동 확장하도록 설정합니다. GCP 로드 밸런서와 연계하면 트래픽 변화에 따라 인스턴스 수가 자동 조정됩니다.

- 실습 (GCP Autoscaler 구성): 앞서 생성한 지역 내 MIG에 대해 autoscaler를 추가합니다. GCP 콘솔의 **인스턴스 그룹 → 자동 확장** 메뉴에서 목표 지표(예: 평균 CPU 50% 유지)와 최대/최소 인스턴스 수를 설정합니다. 설정 후 부하를 증가시키면, Cloud Monitoring 지표를 기반으로 인스턴스가 추가됩니다. `gcloud monitoring metrics list` 등을 사용해 메트릭 수집을 확인할 수 있습니다.

- 예상 결과: 부하가 증가하면서 목표 CPU를 넘으면 자동으로 새로운 VM이 기동되며, Cloud Console에서 MIG의 크기가 증가하는 것을 볼 수 있습니다. 부하가 감소하면 인스턴스 수가 축소됩니다. GCP Monitoring 대시보드에서 인스턴스 개수와 CPU 사용률 변화를 시각화할 수 있습니다.

- 오류 해결 팁: GCP에서도 마찬가지로 인스턴스 확장이 일어나지 않으면 **인스턴스 템플릿 오류**, 이미지 제한, 자원 할당 실패 등을 점검해야 합니다. 템플릿에 SSH 키나 방화벽 태그(예: `http-server`)가 누락되지 않았는지, 인스턴스 시작 스크립트가 정상 실행되었는지 확인하세요. 또한 리전/존 선택 시 가용 머신타입이나 디스크 종류가 지원되는지 검토합니다.

3. 모니터링 및 로깅 시스템

서비스 상태를 실시간으로 모니터링하고 이상 징후를 자동으로 경고 받도록 설정하는 것은 안정적인 운영의 필수 요소입니다. AWS는 **CloudWatch**, GCP는 **Cloud Monitoring & Logging**(이전 Stackdriver)을 통해 서버 및 애플리케이션 모니터링을 제공합니다.

- **AWS CloudWatch:** CloudWatch는 서버(EC2), 데이터베이스(RDS), 네트워크 등 다양한 AWS 리소스의 기본 지표(CPU, 메모리, 네트워크 등)를 수집합니다. 또한 애플리케이션 로그를 **CloudWatch Logs**로 전송하고, **지표(Metric)**를 생성하거나 **경보(Alarm)**를 설정할 수 있습니다. 예를 들어 `aws cloudwatch put-metric-alarm --alarm-name HighCPU --metric-name CPUUtilization --namespace AWS/EC2 --statistic Average --period 300 --threshold 70 --comparison-`

```
operator GreaterThanThreshold --dimensions
```

```
Name=InstanceId,Value=i-1234567890abcdef0 --evaluation-periods 2 --alarm-actions arn:aws:sns:region:account-id:MyTopic
```

처럼 CLI로 임계치 경보를 생성할 수 있습니다.

- 커스텀 메트릭 및 알람: Node.js 서버에서 처리량, 응답시간 등 **사용자 정의 지표**를 CloudWatch에 전송하려면 CloudWatch Agent나 AWS SDK (`PutMetricData` API)를 사용할 수 있습니다. 예를 들어 Node.js 앱에서 다음과 같이 매 초 가짜 트랜잭션 수를 메트릭으로 보내는 코드를 추가할 수 있습니다:

```
const AWS = require('aws-sdk');
const cloudwatch = new AWS.CloudWatch({region: 'ap-northeast-2'});
setInterval(() => {
  cloudwatch.putMetricData({
    Namespace: 'MyApp',
    MetricData: [{ MetricName: 'Transactions', Value: Math.random()*100 }]
  }, (err,data) => { if (err) console.error(err); });
}, 1000);
```

이렇게 생성한 지표를 기반으로 CPU 외에 애플리케이션 레벨의 경보(예: 트랜잭션 200개 초과 시 경보)를 CloudWatch 알람 정책으로 만들 수 있습니다.

- Slack/이메일 알람: CloudWatch 경보는 SNS 토픽과 연결하여 이메일, SMS, 또는 AWS Chatbot을 통한 Slack 알람으로 전송할 수 있습니다. 실제로 CloudWatch Alarm을 SNS 주제와 연계하면, 해당 주제가 **Amazon Chatbot (Q)**에 연결된 경우 Slack 채널로 경고 메시지를 받을 수 있습니다 ¹³. 예를 들어 `aws sns create-topic --name MyAlerts`로 토픽을 만들고, Chatbot에서 해당 토픽을 Slack 워크스페이스에 구독시킨 뒤, 경보의 `--alarm-actions`로 SNS 토픽 ARN을 지정하면 Slack으로 알람이 전송됩니다.
- 실습: CloudWatch 콘솔에서 “경보(Alarms) 생성”을 선택하고, CPUUtilization이나 앞서 보낸 커스텀 지표를 기반으로 임계치 경보를 만듭니다. 알람 대상에는 이메일 주소를 설정하거나 SNS로 Slack/메시징 통합을 구성합니다. 경보가 발생하면 지정한 이메일로 알람이 오거나, Slack 채널에 메시지가 자동 게시되는 것을 확인합니다.
- 오류 해결 팁: 알람이 오지 않을 때는 SNS 구독 상태를 확인하세요. SNS 구독에서 이메일을 활성화했는지, Slack Webhook이 올바른지 점검합니다. 또한 CloudWatch 에이전트 설치 실패나 IAM 권한 부족으로 커스텀 지표가 전송되지 않을 수도 있습니다. 필요한 역할(AmazonEC2RoleforCloudWatch)과 정책이 부여되었는지 확인하세요.
- **GCP Monitoring & Logging**: GCP는 Cloud Monitoring을 통해 VM, 컨테이너, Cloud SQL 등의 메트릭을 수집하고, Cloud Logging을 통해 애플리케이션 로그를 처리합니다. Monitoring에서 정의된 경보 정책은 로그와 지표 데이터를 모두 활용할 수 있습니다. 예를 들어, 메트릭 기반 경보를 만들어 Compute Engine의 CPU 사용률이 일정 수준을 초과할 때 알람을 보내도록 할 수 있습니다.

- 경보 및 알람 채널: GCP 경보 정책(Alerting Policy)은 하나 이상의 **알람 채널(Notification Channel)**과 연결됩니다. 알람 채널로 이메일이나 Slack, SMS 등을 지정할 수 있습니다. 문서 예시처럼 “지원팀 이메일 주소”와 “Slack 채널(#my-support)”을 알람 채널로 동시에 설정할 수 있습니다 ⁹. 실제로 GCP 콘솔에서 경고 정책 생성 시 Slack 통합(Webhook URL)을 연결해 두면, 해당 Slack 워크스페이스로 알람이 전송됩니다.
- 로그 기반 모니터링: Cloud Logging에서는 **로그 기반 지표(log-based metrics)**를 만들어 특정 로그 메시지 출현 빈도를 수집할 수 있습니다. 예를 들어 Node.js 에러 로그가 10분 동안 50회 이상 발생하면 알람을 보낼 수 있습니다. GCP 문서에 따르면 “로그 기반 지표를 사용하여 경보 정책을 생성”할 수 있으며, 로그 내 특정 패턴(예: "ERROR") 출현 횟수에 따라 알람을 받을 수 있습니다 ¹⁴.
- 실습: 간단한 예로, Node.js 앱에서 `console.error("High latency!")`를 일정 빈도로 출력하게 한 뒤, GCP 로그 뷰어에서 이 메시지가 들어간 로그 항목을 검색하여 **로그 기반 지표**를 생성합니다. 이후 Cloud

Monitoring에서 “로그 기반 경고 정책”을 만들어 이 지표가 일정 임계값을 초과하면 Slack/이메일로 알림하도록 합니다. 이때 GCP에서 Slack 알림은 Webhook을 만들어 연결하거나, Cloud Functions 및 Pub/Sub을 활용해 Slack으로 메시지를 보내는 방식으로 구현할 수 있습니다.

- 오류 해결 팁: GCP 경보가 동작하지 않는다면 알림 채널 설정을 확인하세요. Slack Webhook URL이 올바르게 입력되었는지, email 수신자가 실제 구글 계정인지 점검합니다. 또한, 로그 기반 경보를 위한 로그가 실제로 수집되고 있는지 Logging 대시보드에서 로그가 존재하는지 먼저 확인해야 합니다.

4. 종합 프로젝트 및 최적화

지금까지 학습한 내용을 통합하여 실제 **고가용성 웹 서비스** 아키텍처를 설계하고 성능·비용을 최적화하는 연습을 합니다. 예를 들어 다음과 같은 시나리오를 구성할 수 있습니다.

- **아키텍처 설계 실습:** 예시 시나리오로, 소규모 Node.js 웹 애플리케이션을 구축한다고 가정합니다. AWS 쪽에서는 EC2 Auto Scaling 그룹과 ALB, RDS Multi-AZ를 이용하여 수평적 확장과 데이터베이스 이중화를 구현하고, GCP 쪽에서는 Managed Instance Group과 글로벌 HTTP 로드 밸런서, Cloud SQL(프라이빗 IP, 다중 영역 옵션) 등을 사용합니다. 두 환경 모두 각 AZ/리전에 최소 2대 이상의 서버를 배포하고, 마지막으로 **Cross-Region Disaster Recovery**를 위해 콘텐츠를 S3-및 GCS-저장소에 백업하거나 데이터베이스 백업을 다른 리전으로 복제해둡니다.
- **CI/CD 배포 실습:** Git 레포지토리에 Node.js 애플리케이션 코드를 push하면 AWS CodePipeline 또는 GitHub Actions, GCP Cloud Build를 통해 자동으로 인프라가 배포되도록 CI/CD 파이프라인을 구성합니다. 예를 들어 AWS CodePipeline을 사용해 GitHub → CodeBuild → CloudFormation 스택 배포 과정을 만들어 보거나, GCP에서는 Cloud Build 트리거로 GitHub 커밋 시 빌드 및 `gcloud` 명령 배포를 수행하게 설정합니다. 이를 통해 코드 변경 시 자동으로 애플리케이션 인스턴스가 업데이트되도록 연계합니다.
- **성능 최적화:** 트래픽 증가에 대비해 캐싱 및 CDN을 사용합니다. 예를 들어 정적 파일은 AWS CloudFront 또는 GCP Cloud CDN을 통해 캐시하고, 세션이 필요 없는 API 서버는 인스턴스 수를 탄력적으로 늘려 처리합니다. Node.js에서는 `cluster` 모듈을 사용해 멀티 스레드를 활용하거나, MySQL 커넥션 풀링 등을 통해 데이터베이스 연결 부하를 줄일 수 있습니다. 또한 프록시 서버(Nginx)를 앞단에 두어 정적 콘텐츠와 API 요청을 분리 처리하면 성능이 개선됩니다.
- **비용 분석 및 절감:** 구축된 인프라의 비용을 모니터링하고 최적화합니다. AWS에서는 **Cost Explorer**를 사용하여 서비스별 월간 요금을 분석하고, GCP에서는 **Cost Management** 대시보드에서 프로젝트별 비용을 확인합니다. 예를 들어 Auto Scaling 그룹의 불필요한 인스턴스 유휴 시간을 줄이기 위해 최소 용량을 조정하거나, 예약 인스턴스·스팟 인스턴스를 도입하여 비용을 절감할 수 있습니다. RDS나 Cloud SQL 인스턴스 사이즈를 실제 트래픽에 맞춰 조정하고, 사용량이 적은 리소스는 종료하여 과금을 줄입니다.
- **종합 프로젝트 실습 시나리오:** 마지막 실습에서는 예를 들어 “전세계 사용자를 대상으로 Node.js 기반 웹 서비스를 운영”하는 환경을 구성하고, 다음 단계를 따라해 봅니다:
 - **환경 준비:** 먼저 AWS와 GCP 콘솔에 접근하여 필요한 권한(IAM, 서비스 계정 권한)을 확인합니다. 프로젝트/계정을 생성하고 리소스 네이밍 컨벤션을 정합니다. 각 클라우드의 서비스 사용량 한계(quota)를 확인하고 필요한 경우 증액합니다.
 - **인프라 배포:** 이전 단계에서 계획한 아키텍처에 따라 리소스를 생성합니다. AWS의 경우 CDK/CloudFormation 또는 CLI로 VPC, 서브넷, 보안그룹, EC2/ELB/ASG, RDS 다중 AZ 등을 순서대로 생성합니다. GCP의 경우 Deployment Manager 또는 Terraform으로 네트워크, VM 인스턴스 그룹, 로드 밸런서, Cloud SQL을 설정합니다.
 - **애플리케이션 배포:** Node.js 애플리케이션 코드를 Git 저장소에 커밋하고, CI/CD 파이프라인을 통해 각 클라우드에 배포합니다. 배포 완료 후 웹 브라우저나 `curl`로 ELB/글로벌 LB의 주소에 접근하여 “Hello World” 메시지가 나오는지 확인합니다.
 - **부하 테스트:** `ab`, `wrk`, `locust` 등 부하 도구로 다량의 요청을 시뮬레이션하여 Auto Scaling이 작동하는지 확인합니다. 이때 인스턴스 수가 늘어나고, CPU/응답시간 메트릭이 안정화되는지 모니터링합니다.

- **모니터링 설정:** CloudWatch와 Cloud Monitoring에서 CPU, 메모리, 네트워크, 응답시간 등의 지표 대시보드를 만듭니다. 경보를 설정해 임계치 이상일 때 이메일/Slack 알림이 가도록 합니다.
- **문제 해결 연습:** 예를 들어 하나의 AZ 인스턴스를 제거하거나, DB 연결을 끊는 등의 장애 상황을 시뮬레이션하여 서비스 연속성을 확인합니다. 장애 복구(Failover)가 제대로 이루어지지 않을 경우 각 로그와 경보를 참고해 원인을 분석합니다.
- **성과 분석 및 개선:** 최종적으로 트래픽 패턴과 비용 데이터를 기반으로 인프라를 최적화합니다. 예를 들어 평균 트래픽이 적은 야간에는 최소 인스턴스를 줄이고, 사용량 급증을 대비해 예약 인스턴스 대신 스팟 인스턴스 활용을 고려합니다. 애플리케이션 코드 최적화(CPU 사이클 절약, 메모리 누수 수정)도 함께 검토합니다.

이상의 과정을 통해 학습자는 Node.js 기반의 웹 애플리케이션을 대상으로 AWS와 GCP 양쪽에서 고가용성 아키텍처를 구성하고, 실제 트래픽 부하와 모니터링 환경에서 서비스의 안정성을 검증해 볼 수 있습니다. 각 단계에서 이론과 연계된 실습을 통해 체계적으로 따라할 수 있도록 안내하면, 클라우드 기초 수준의 수강자도 충분히 실무 적용이 가능한 노하우를 쌓을 수 있을 것입니다.

참고자료: AWS 및 Google Cloud 공식 문서와 기술 블로그를 참조하여 내용을 구성하였습니다 1 15 2 4 13 9 14 .

1 Amazon RDS 다중 AZ 배포 | 클라우드 관계형 데이터베이스 | Amazon Web Services

<https://aws.amazon.com/ko/rds/features/multi-az/>

2 11 웹 서버 로드 밸런싱 | 서버 로드 밸런싱 | Amazon Web Services

<https://aws.amazon.com/ko/elasticloadbalancing/>

3 Multi-regional deployment on Compute Engine | Cloud Architecture Center | Google Cloud

<https://cloud.google.com/architecture/multiregional-vms>

4 12 Cloud Load Balancing overview | Google Cloud

<https://cloud.google.com/load-balancing/docs/load-balancing-overview>

5 6 7 8 10 15 AWS 기반 재해 복구(DR) 아키텍처, 1부: 클라우드에서의 재해 복구 전략 | AWS 기술 블로그

<https://aws.amazon.com/ko/blogs/tech/disaster-recovery-dr-architecture-on-aws-part-i-strategies-for-recovery-in-the-cloud-1/>

9 Alerting overview | Cloud Monitoring | Google Cloud

<https://cloud.google.com/monitoring/alerts>

13 Test notifications from AWS services to chat channels using CloudWatch - Amazon Q Developer in chat applications

<https://docs.aws.amazon.com/chatbot/latest/adminguide/test-notifications-cw.html>

14 Monitor your logs | Cloud Logging | Google Cloud

<https://cloud.google.com/logging/docs/alerting/monitoring-logs>